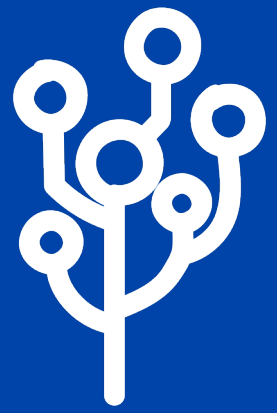
Branching und Merging Übersicht



Warum Branching?

- In einer kollaborativen Umgebung ist es üblich, dass mehrere Entwickler an denselben Quellcode arbeiten. Während einige Entwickler Fehler beheben, werden andere neue Funktionen implementieren, usw.
- Da so viel los ist, muss ein System vorhanden sein, mit dem verschiedene Versionen derselben Codebasis verwaltet werden können.
- Durch das Verzweigen (Branching) kann jeder Entwickler von der ursprünglichen Codebasis abweichen und seine Arbeit von anderen isolieren.
- Es hilft Git auch, Versionen später einfach zusammenzuführen.

Beispiel für einfaches Branching und Merging

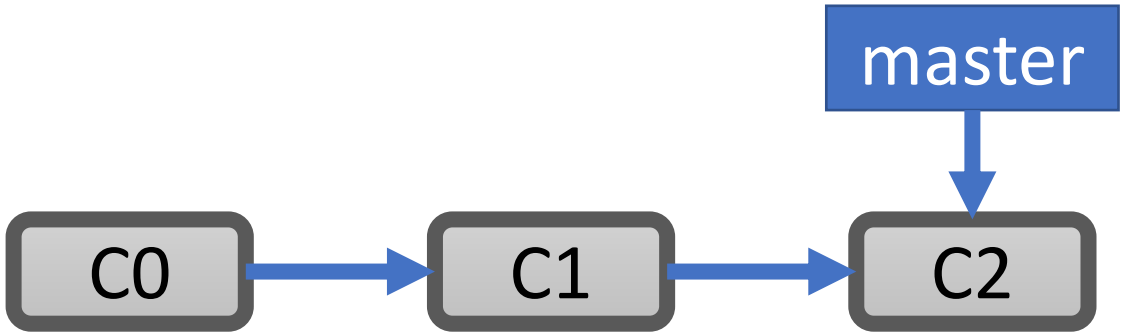
- Lass uns das Ganze an einem praxisnahen Beispiel durchgehen, dessen Workflow zum Thema Branching und Zusammenführen du im echten Leben verwenden kannst:
  - Arbeite an einem Projekt (Webseite, Analyse, Programm)
  - Erstelle einen (Zweig) Branch für ein neues Feature oder neue Geschichte (Story), an der du arbeitest.
  - Arbeite danach in diesem neuen Branch.

Beispiel für einfaches Branching und Merging

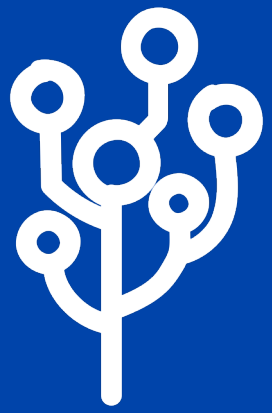
- In diesem Augenblick kommt ein Anruf, dass ein kritisches Problem aufgetreten ist und sofort gelöst werden muss. Du machst folgendes:
  - Wechsle zu Deinem „Produktiv“-Zweig (master-Branch) zurück.
  - Erstelle einen neuen Branch für den Hotfix.
  - Nach dem Testen führst Du den Hotfix-Branch mit dem „Produktiv“-Branch zusammen.
  - Wechsle wieder in Deine alte Arbeit zurück und arbeite dort weiter.

Beispiel für einfaches Branching und Merging

- Du arbeitest an deinem Projekt und hast bereits einige Commits durchgeführt:

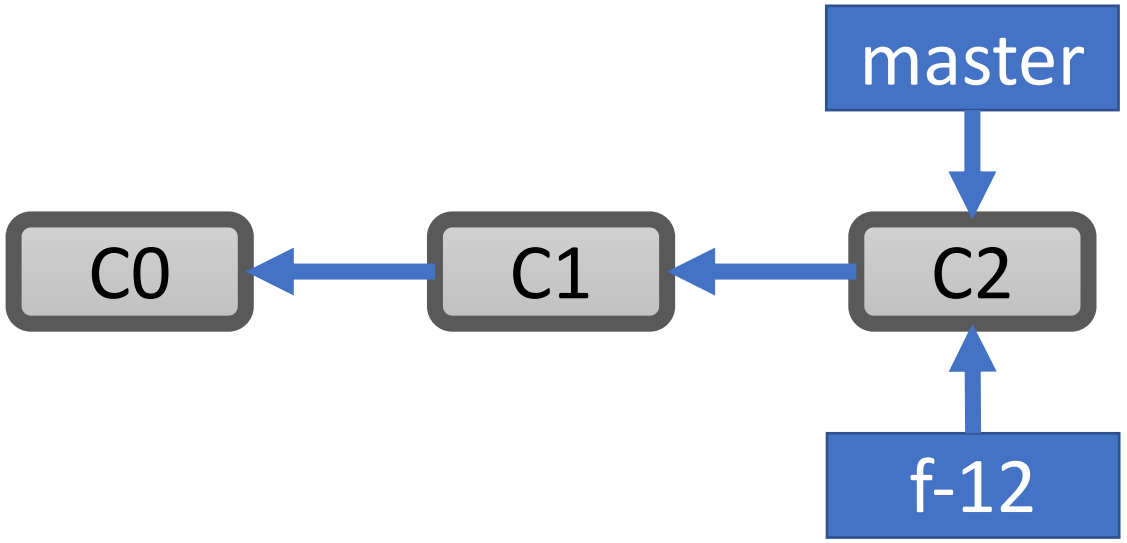


Neuen Branch erstellen



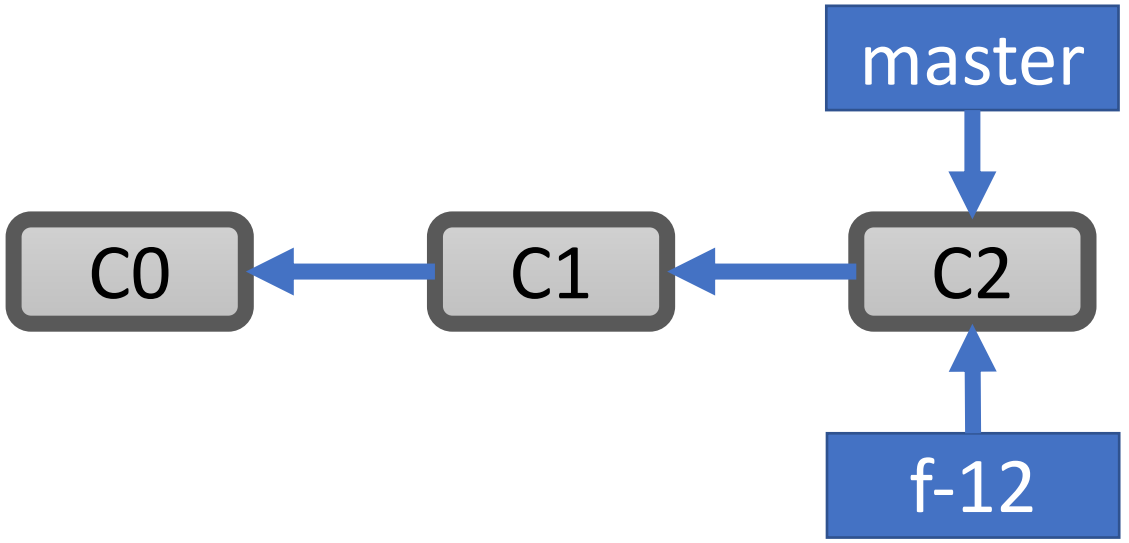
Beispiel für einfaches Branching und Merging

- Als nächstes willst du eine neue Erweiterung deiner Anwendung entwickeln. Dafür erstellst du zunächst einen neuen Feature-Branch **f-12:**



Beispiel für einfaches Branching und Merging

- Als nächstes willst du eine neue Erweiterung deiner Anwendung entwickeln. Dafür erstellst du zunächst einen neuen Feature-Branch **f-12:**



Du kannst die ID aus einem Issuetracker (z.B. Jira) für den Branch verwenden.

Beispiel für einfaches Branching und Merging

- In der Praxis kann der Name noch ausführlicher angegeben werden:
  - Feature-ProjectID-IssueItrackerID-Beschreibung
  - feature-abc-12-Userlogin
- Verwende **git checkout** mit der Option **-b** um einen neuen Branch zu erstellen:

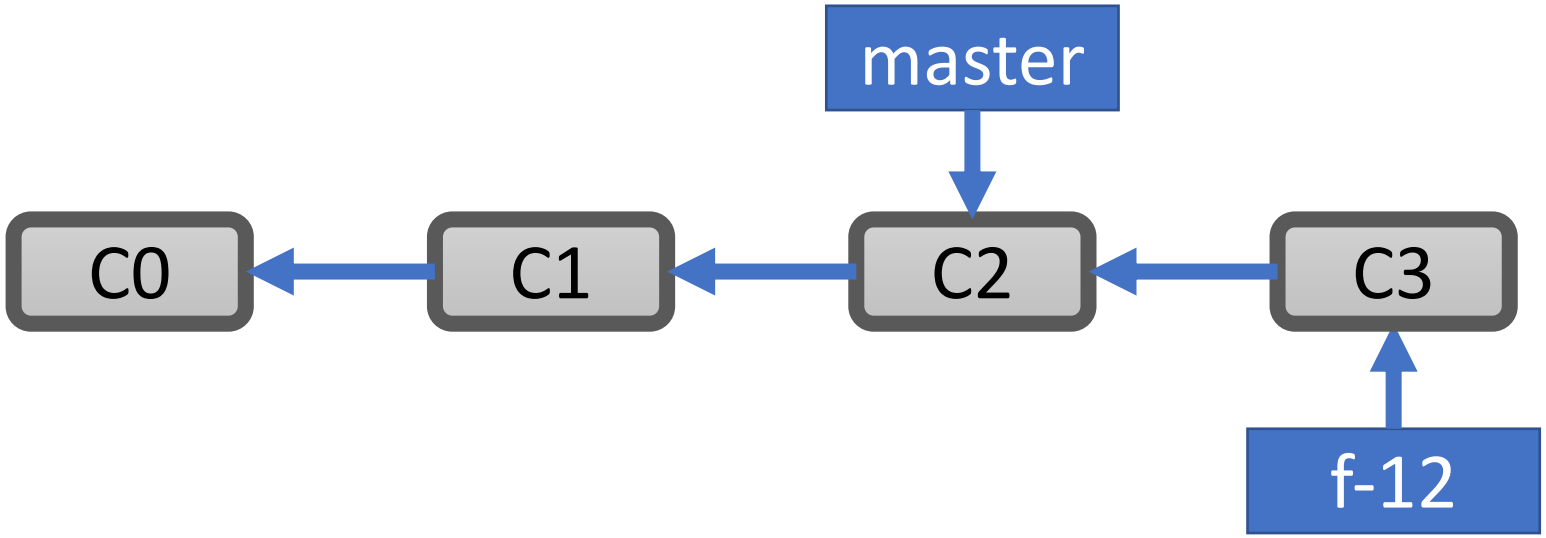
  git checkout -b feature-abc-12-Userlogin
- Das war die Abkürzung von

  git branch feature-abc-12-Userlogin

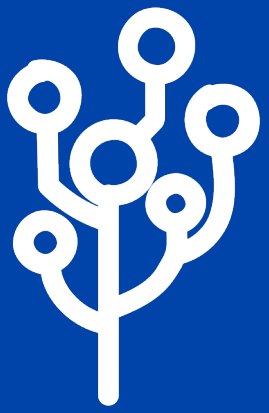
  git checkout feature-abc-12-Userlogin

Beispiel für einfaches Branching und Merging

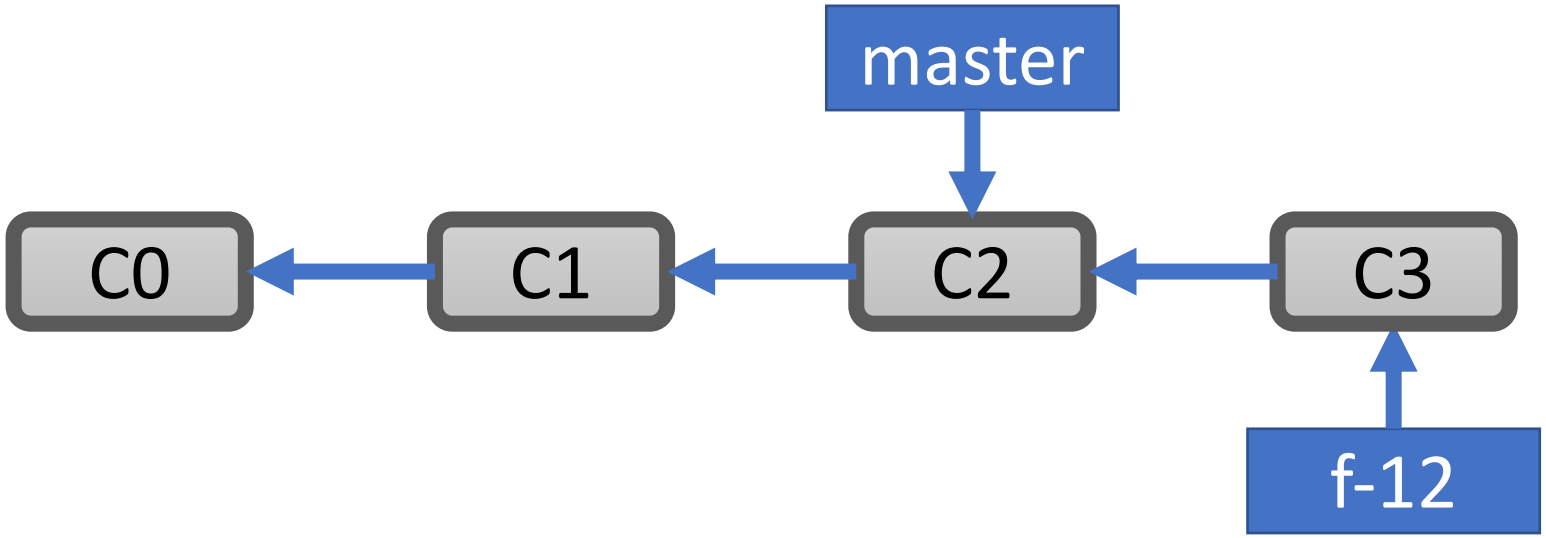
- Nachdem du den Branch erstellt hast, kannst arbeitest du an deinem neuen Feature für die Anwendung und machst einen Commit. Das bewegt dann den **f-12**-Branch vorwärts:



Branch wechseln (hotfix)



Beispiel für einfaches Branching und Merging



Beispiel für einfaches Branching und Merging

- Nun bekommst du einen Anruf, in dem dir mitgeteilt wird, dass es ein Problem mit der Anwendung gibt, das du umgehend beheben musst.
- Jetzt bist du erleichtert, dass du Git verwendest:
  - Bei Git musst du deine Fehlerkorrektur nicht zusammen mit den **f12-Änderungen** einbringen.
  - Und du musst keine Zeit damit verschwenden, deine bisherigen Änderungen rückgängig zu machen, bevor du mit der Fehlerbehebung an der Produktionsumgebung beginnen kannst.

Beispiel für einfaches Branching und Merging

- Alles, was du tun musst, ist, zu deinem **master-Branch** wechseln.
- Beachte jedoch, dass dich Git den Branch nur wechseln lässt, wenn bisherige Änderungen in deinem Arbeitsverzeichnis oder deiner Staging-Area nicht in Konflikt mit dem Branch stehen, zu dem du nun wechseln möchtest.
- Am besten ist es, wenn ein sauberer Arbeitsstand vorliegt, wenn man den Branch wechselt. Wir werden uns später mit Wegen befassen, dieses Verhalten zu umgehen (namentlich „Stashing“).

Beispiel für einfaches Branching und Merging

- Vorerst aber hast du deine Änderungen bereits comitted, sodass du zu deinem **master-Branch** zurückwechseln kannst:
  - **git checkout master**

- Zu diesem Zeitpunkt befindet sich das Arbeitsverzeichnis des Projektes in exakt dem gleichen Zustand, in dem es sich befand, als du mit der Arbeit an dem **Feature 12** begonnen hast und du kannst dich direkt auf Deinen Hotfix konzentrieren.

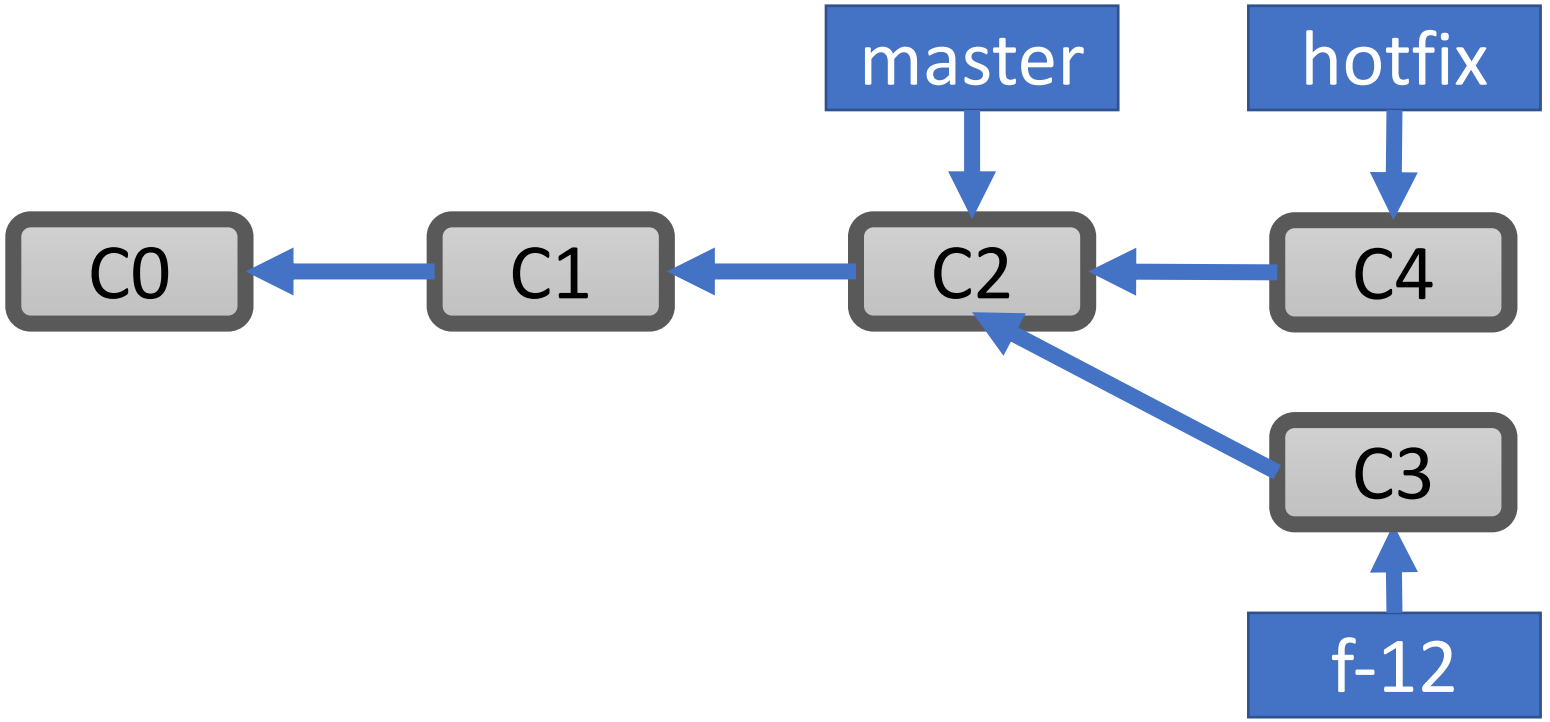
Beispiel für einfaches Branching und Merging

- Dies ist ein wichtiger Moment, um sich vor Augen zu halten, dass Git Dein **Arbeitsverzeichnis** auf den Zustand des Commits, auf den dieser Branch zeigt, zurücksetzt.
- Es erstellt, entfernt und verändert Dateien automatisch, um sicherzustellen, dass Deine Arbeitskopie haargenau so aussieht wie der Zweig nach Deinem letzten Commit.

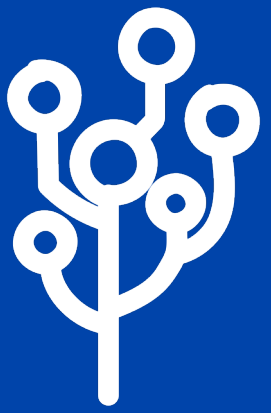
Beispiel für einfaches Branching und Merging

- Nun hast du einen Hotfix zu erstellen. Lass uns dazu einen hotfix-Branch erstellen, an dem du bis zu dessen Fertigstellung arbeitest:
  - **git checkout -b hotfix**
  - **git commit -m „fix docker file“ Dockerfile**

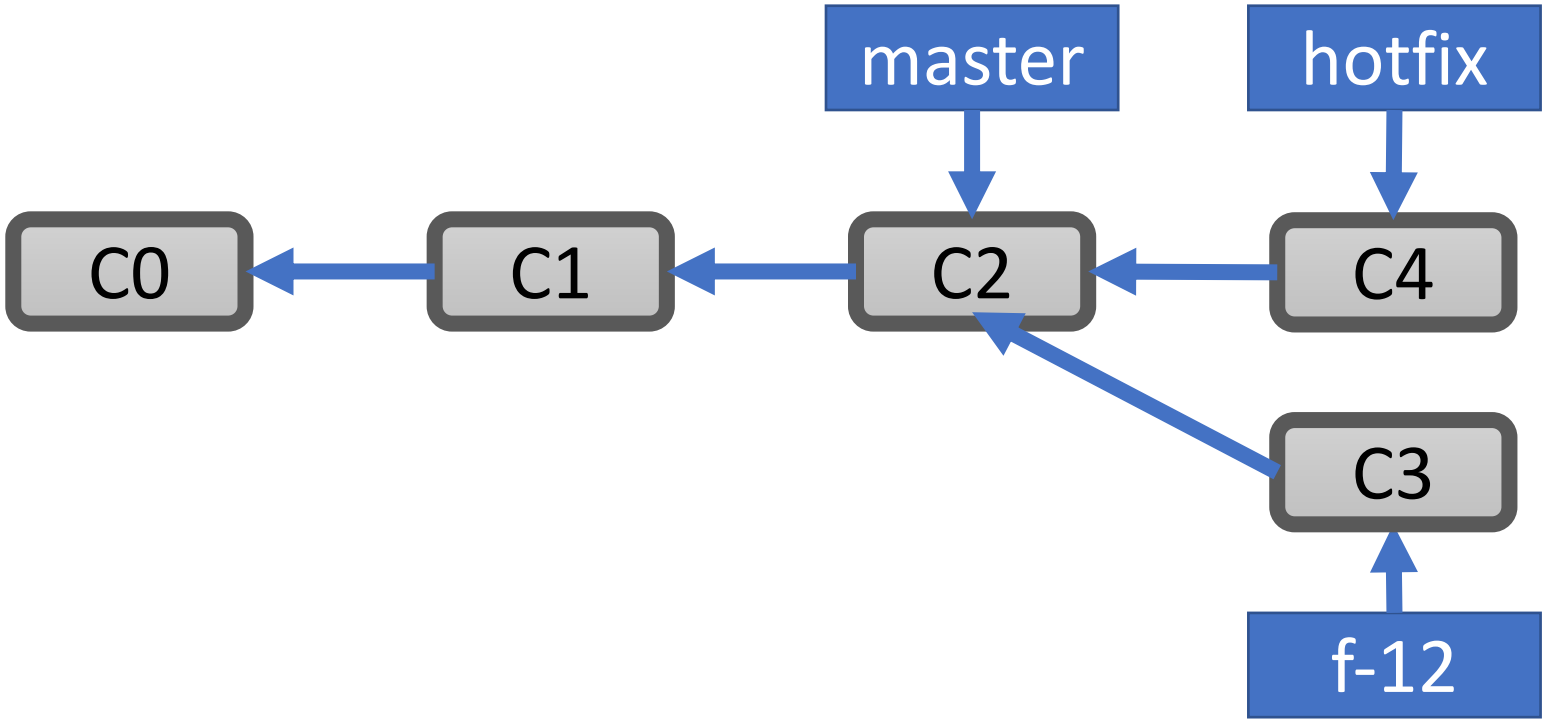
Beispiel für einfaches Branching und Merging



Merge (hotfix)



Beispiel für einfaches Branching und Merging



Beispiel für einfaches Branching und Merging

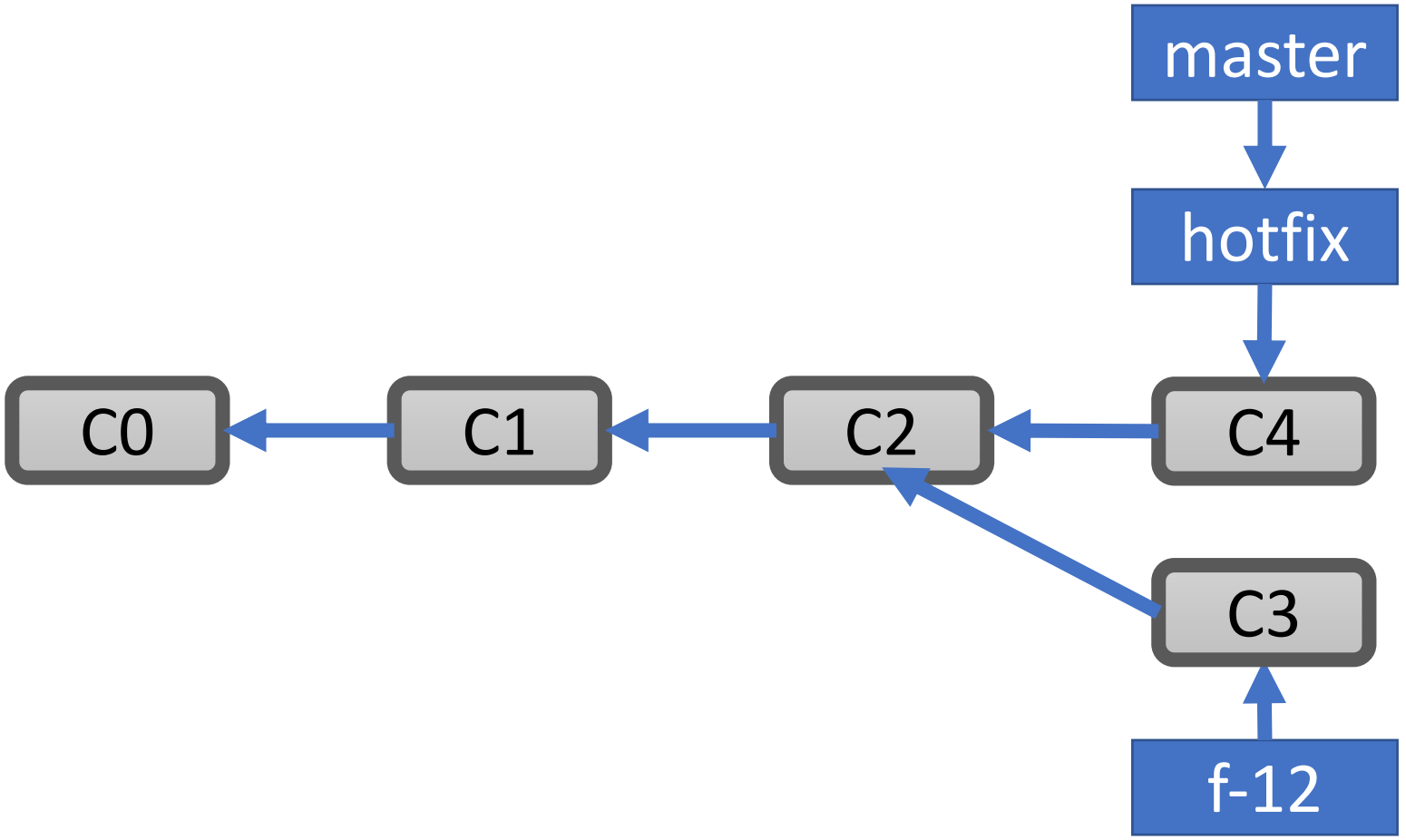
- Normalerweise führst du deine Tests durch und stellst sicher, dass der Hotfix seinen Zweck erfüllt.
- Danach wirst du ihn dann mit dem **master-Branch** zusammenführen, um ihn in die Produktionsumgebung zu integrieren. Das machst Du mit der Anweisung **git merge**:

  - **git checkout master**
  - **git merge hotfix**

Beispiel für einfaches Branching und Merging

- Du wirst die Mitteilung **„Fast Forward“** während des Zusammenführens bemerken. Da der neue Commit direkt von dem ursprünglichen Commit abstammt, auf den sich der nun eingebrachte Zweig bezieht, bewegt Git einfach den Zeiger weiter.
- Deine Modifikationen befinden sich nun als Schnappschuss in dem Commit, auf den der **master-Branch** zeigt, diese lassen sich nun veröffentlichen.

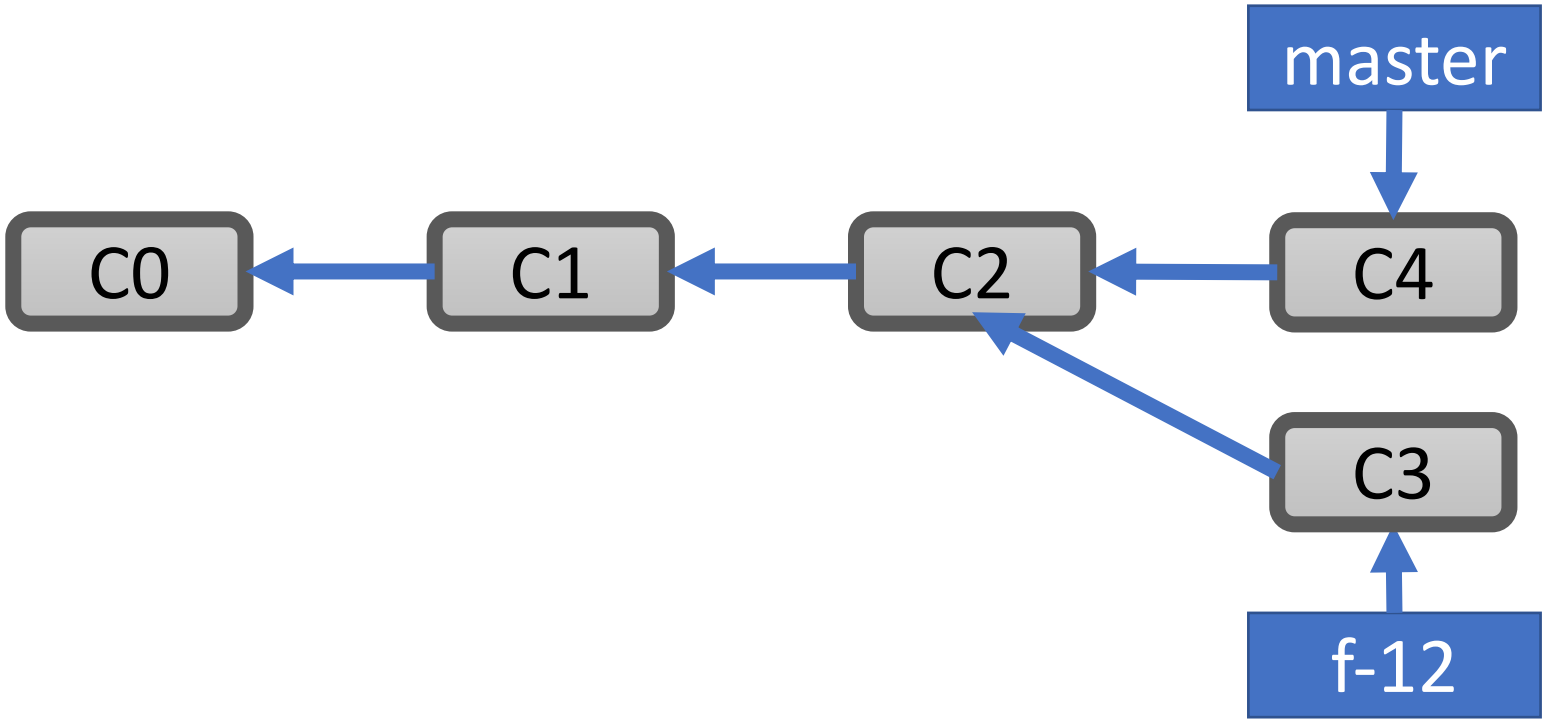
Beispiel für einfaches Branching und Merging



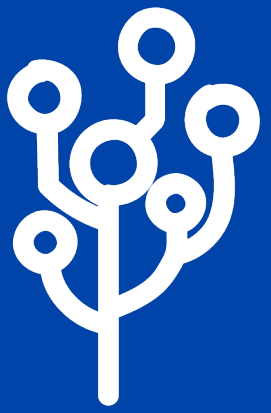
Beispiel für einfaches Branching und Merging

- Nachdem dein superwichtiger Hotfix veröffentlicht wurde, kannst du dich wieder deiner ursprünglichen Arbeit zuwenden.
- Vorher wird sich allerdings des nun nutzlosen hotfix-Zweiges entledigt, schließlich zeigt der **master-Branch** ebenfalls auf die aktuelle Version.
- Du kannst ihn mit der **-d**-Option von **git branch** entfernen:
  - **git branch -d hotfix**

Beispiel für einfaches Branching und Merging



Merge (feature)



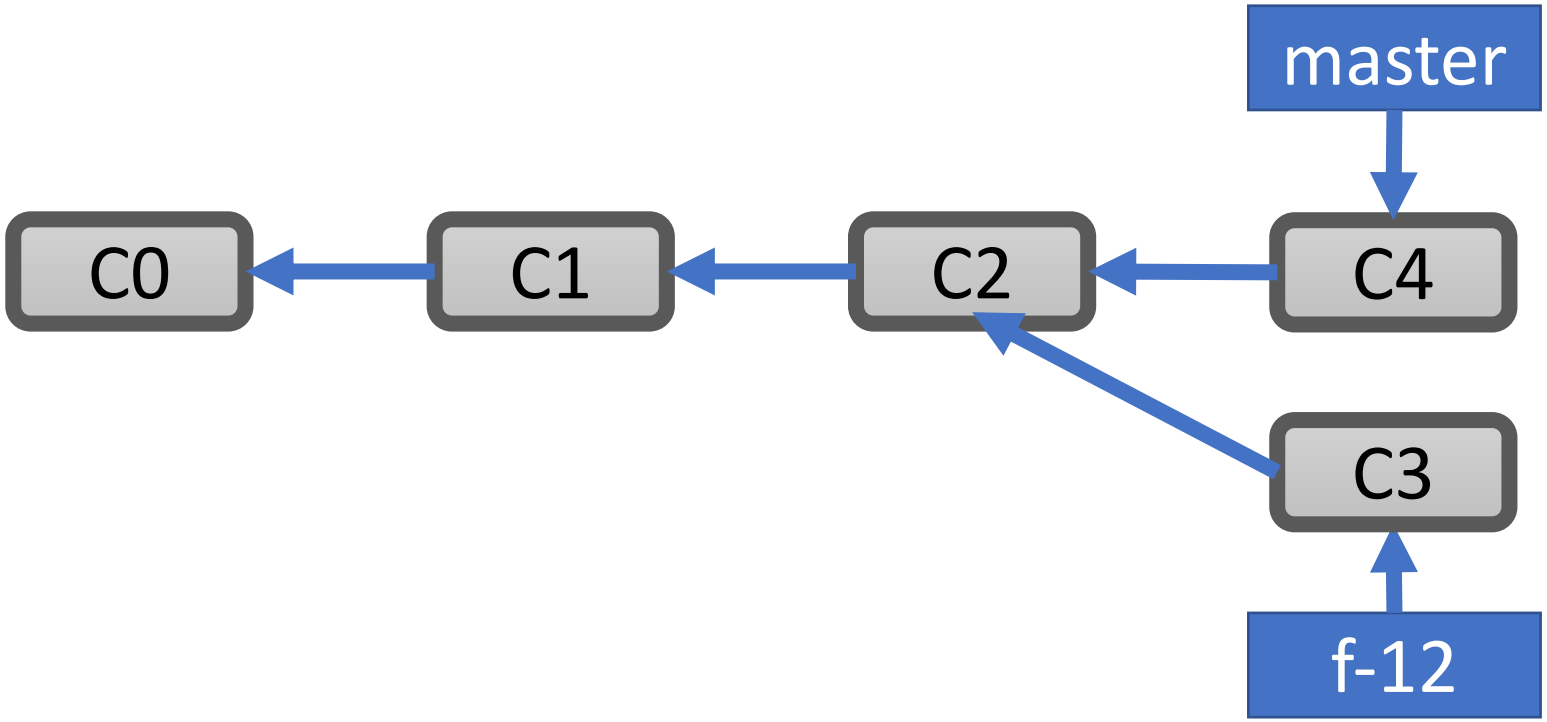
Branching Feature

- Nun kannst du zu deinem **f-12**-Branch zurückwechseln und mit deiner Arbeit **fortfahren**:
  - **git checkout feature-abc-12-Userlogin**

Branching Feature

- An dieser Stelle ist anzumerken, dass die Änderungen an dem hotfix-Branch nicht in deinen **f-12**-Branch eingeflossen sind.
- Falls nötig, kannst du den master-Branch allerdings mit der Anweisung **git merge master** mit deinem Branch kombinieren.
- Oder du wartest, bis du den **f-12**-Branch später in den master-Branch zurückführst.

Branching Feature



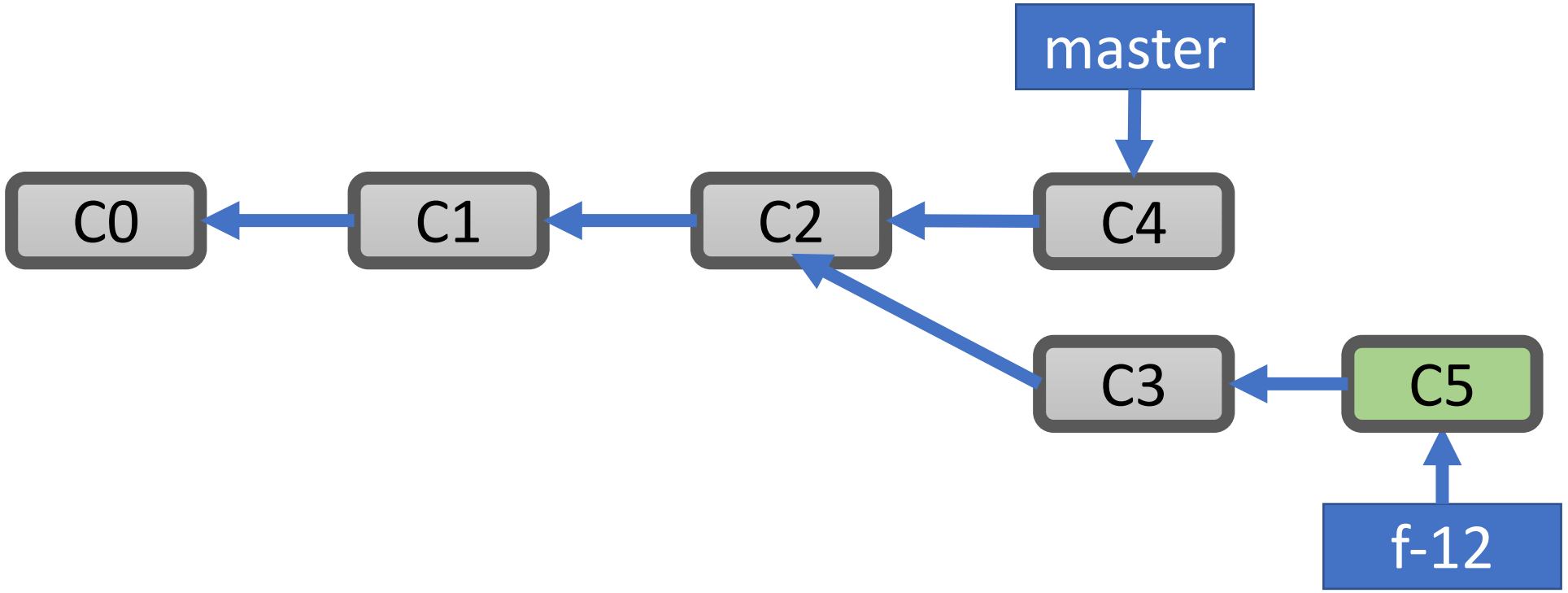
Branching Feature

- Angenommen, du hast entschieden, dass deine Arbeiten an **f-12** abgeschlossen sind und das Ganze soweit ist, dass es mit dem **master-Branch** zusammengeführt werden kann.
- Um das zu erreichen, wirst du deinen **f-12** Branch in den **master-Branch** einfließen lassen, genauso wie du es mit dem **hotfix-Branch** zuvor getan hast.

Branching Feature

- Du musst nur mit der Anweisung **checkout** zum dem Branch zu wechseln, in welchen du etwas einfließen lassen möchtest und dann die Anweisung **git merge** ausführen:
  - **git checkout master**
  - **git merge feature-abc-12-Userlogin**

Branching Feature



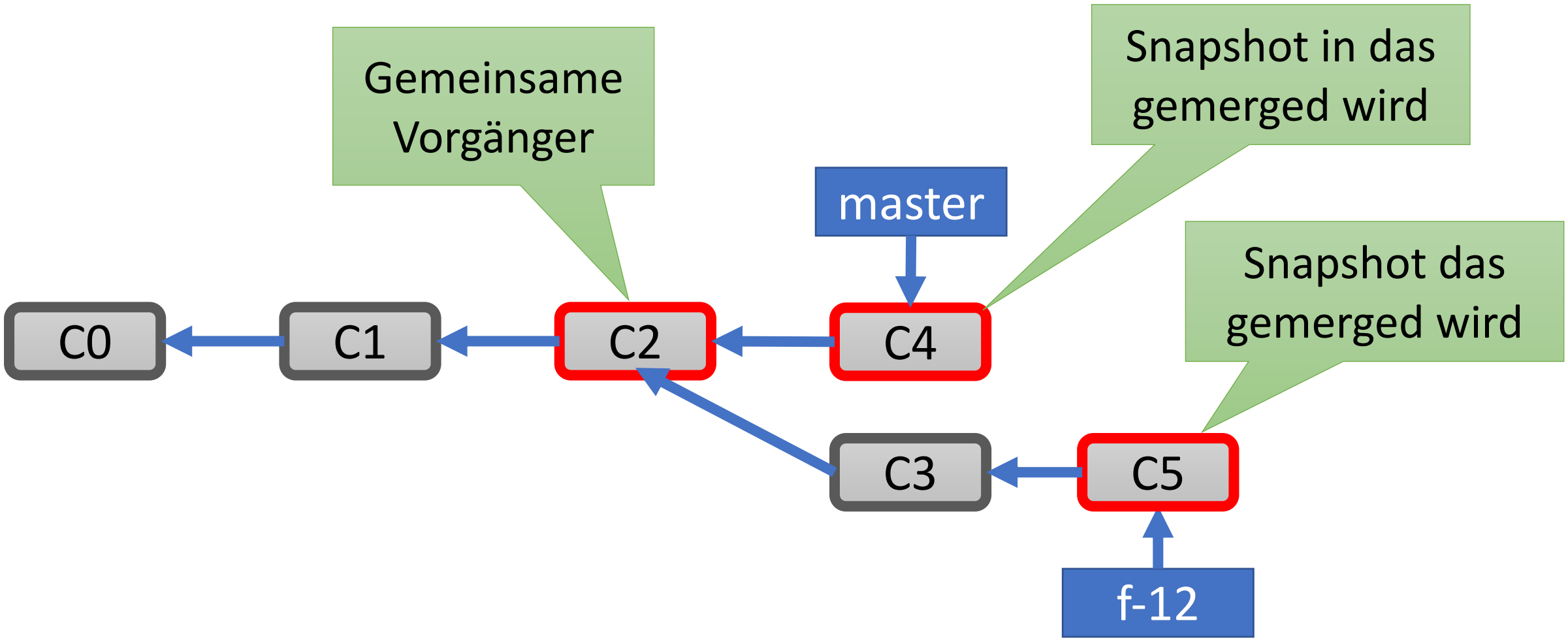
Branching Feature

- Das sieht ein bisschen anders aus als das **hotfix merge** von vorhin.
- Hier haben sich die Entwicklungsstränge schon zu einem früheren Zeitpunkt geteilt. Da der **commit** auf dem Branch, in dem du dich befindest, kein direkter Nachfolger von dem Branch ist, in den du das merge machst, hat Git einiges zu tun.

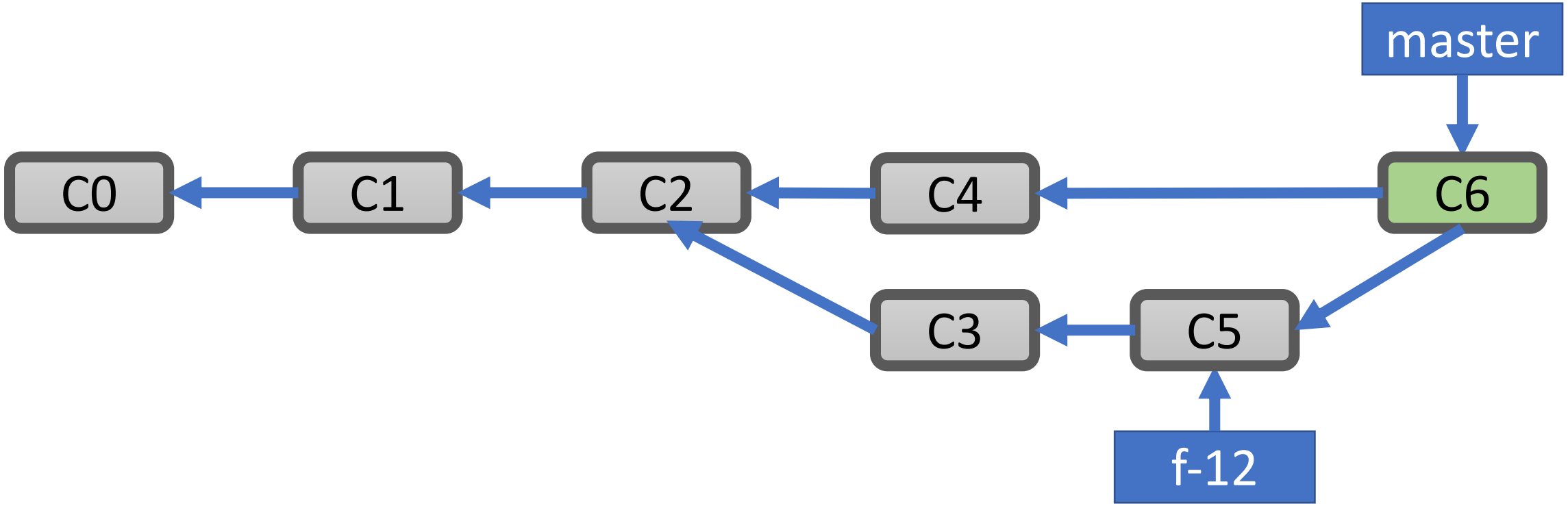
Branching Feature

- Dazu macht Git einen 3-Wege merge, wobei es die **beiden Schnappshots** verwendet, welche auf die Enden der Branches zeigen, und den **gemeinsamen Vorfahren** dieser beiden.
- Git erkennt dabei automatisch den am besten geeigneten gemeinsamen Vorgänger für die Branchzusammenführung.

Beispiel für einfaches Branching und Merging



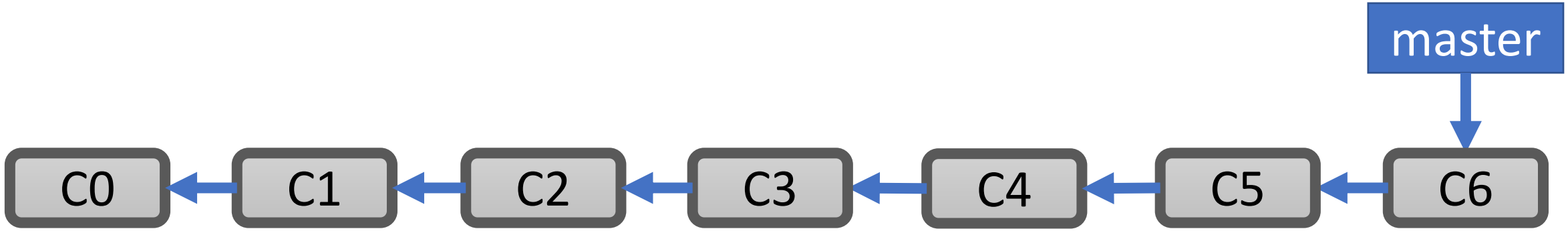
Beispiel für einfaches Branching und Merging



Branching Feature

- Jetzt, da wir die Arbeit zusammengeführt haben, ist der **f-12**-Branch nicht mehr notwendig. Du kannst ihn löschen (und das Ticket im Ticket-Tracking-System schließen).
  - **git branch -d feature-abc-12-Userlogin**

Beispiel für einfaches Branching und Merging



Challenge!



Challenge!

- Deine Kollegen sind mittlerweile begeistern von deinen Git Kenntnissen und kommen auf dich zu um ihren Branch zu mergen. Sie bekommen es einfach nicht hin.
- Du hilfst ihnen natürlich gerne. ☺



Aufgabenbeschreibung

- “Forke“ und klone das Flair Projekt von Zalando von dieser Seite (oder aus den Anlagen dieser Lektion):
  - https://github.com/datamics/flair
- Schaue dir die bestehenden Zweige an (**git branch –a**)
- Zeige Änderungen mit (**git diff master feature-101)** an (optional)
- Merge den **hotfix**-Branch in den **master**-Branch
- Merge den **feature-101-**Branch in den **master-**Branch
- Lade die Änderungen auf das Remote-Repository

Challenge!

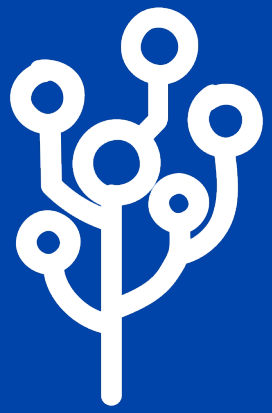
Lösung



Aufgabenbeschreibung

- “Forke“ und klone das Flair Projekt von Zalando von dieser Seite (oder aus den Anlagen dieser Lektion):
  - https://github.com/datamics/flair
- Schaue dir die bestehenden Zweige an (**git branch –a**)
- Zeige Änderungen mit (**git diff master feature-101)** an (optional)
- Merge den **hotfix**-Branch in den **master**-Branch
- Merge den **feature-101-**Branch in den **master-**Branch
- Lade die Änderungen auf das Remote-Repository

Merge Konflikt



Merge Konflikt

- Gelegentlich verläuft der Prozess nicht ganz so glatt. Wenn Du an den selben Stellen in den selben Dateien unterschiedlicher Branches etwas geändert hast, kann Git diese nicht sauber zusammenführen.

Merge Konflikt

- Wenn Dein Fix an **f-12** die selbe Stelle in einer Datei verändert hat, die Du auch mit **hotfix** angefasst hast, wirst Du einen **'merge**'-Konflikt erhalten, der ungefähr so aussehen könnte:
  - **git merge feature-abc-12-Userlogin**

    Auto-merging Dockerfile

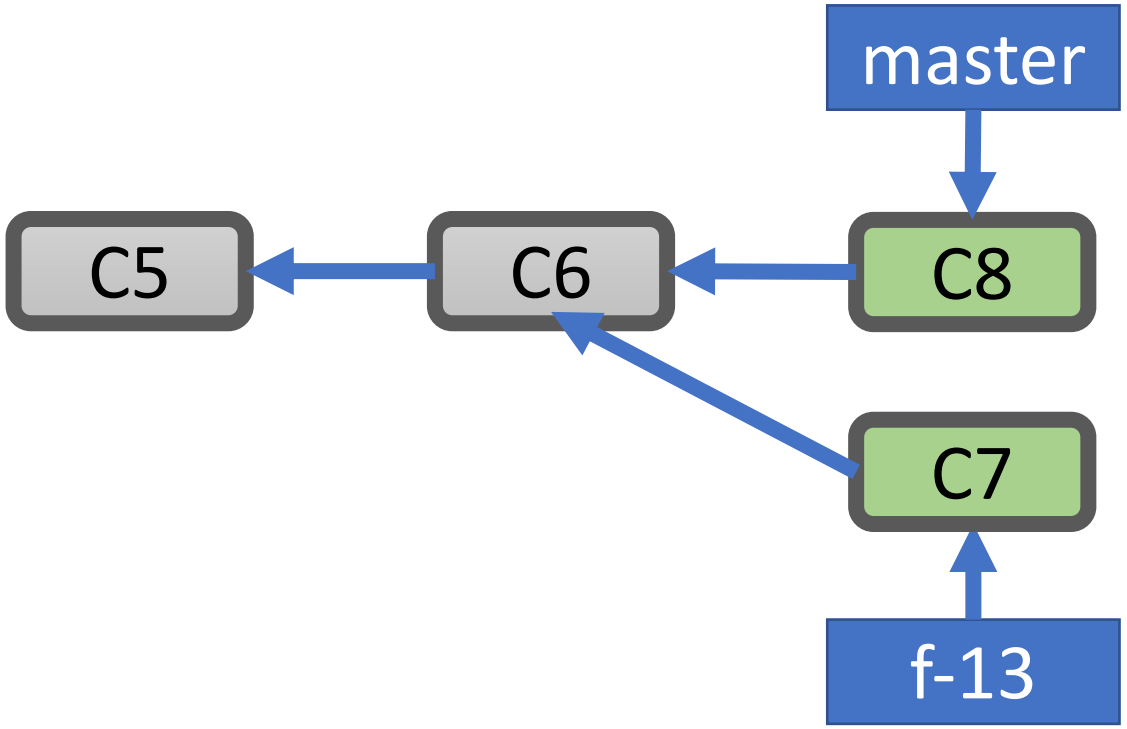
    CONFLICT (content): Merge conflict in Dockerfile

    Automatic merge failed; fix conflicts and then commit the result.

Beipiel Merge Konflikt

- Erstellen wir einen neuen Feature branch
  - **git checkout -b feature-abc-13-Docker**
- Ändern die Dockerfile Datei und führen einen commit mit den Änderungen aus.
  - # 2. Fix Docker
- Danach wechselst du in den master-Branch und änderst dort auch die Dockerfile Datei und führst einen commit aus:
  - # 2. Fix docker file

Beipiel Merge Konflikt



Beipiel Merge Konflikt

- Git hat hier keinen 'merge commit' erstellt. Es hat den Prozess gestoppt, damit Du den Konflikt beseitigen kannst.
- Wenn Du sehen willst, welche Dateien 'unmerged' aufgrund eines 'merge' Konflikts sind, benutze einfach **git status.**

Beipiel Merge Konflikt

- Alles, was einen 'merge' Konflikt aufweist und nicht gelöst werden konnte, wird als 'unmerged' aufgeführt.
- Git fügt den betroffenen Dateien Standard-Konfliktlösungsmarker hinzu, sodass Du diese öffnen und den Konflikt manuell lösen kannst/musst.

Beipiel Merge Konflikt

- Deine Datei enthält einen Bereich, der so aussehen könnte:

```
<<<<<<< HEAD
# 2. Fix Docker
=======
# 2. Fix docker file
>>>>>>> f-13
```

Beipiel Merge Konflikt

- Um den Konflikt zu lösen, musst Du Dich entweder für einen der beiden Teile entscheiden oder du ersetzt den Teil komplett:

  # 2. Fix Docker file

- Diese Lösung hat von beiden Teilen etwas und die Zeilen mit <<<<<<<, =======, und >>>>>>> sind komplett gelöscht.
- Nachdem du alle problematischen Bereiche in allen von dem Konflikt betroffenen Dateien beseitigt hast, führe einfach **git add** für alle betroffenen Dateien aus und markiere sie damit als bereinigt.

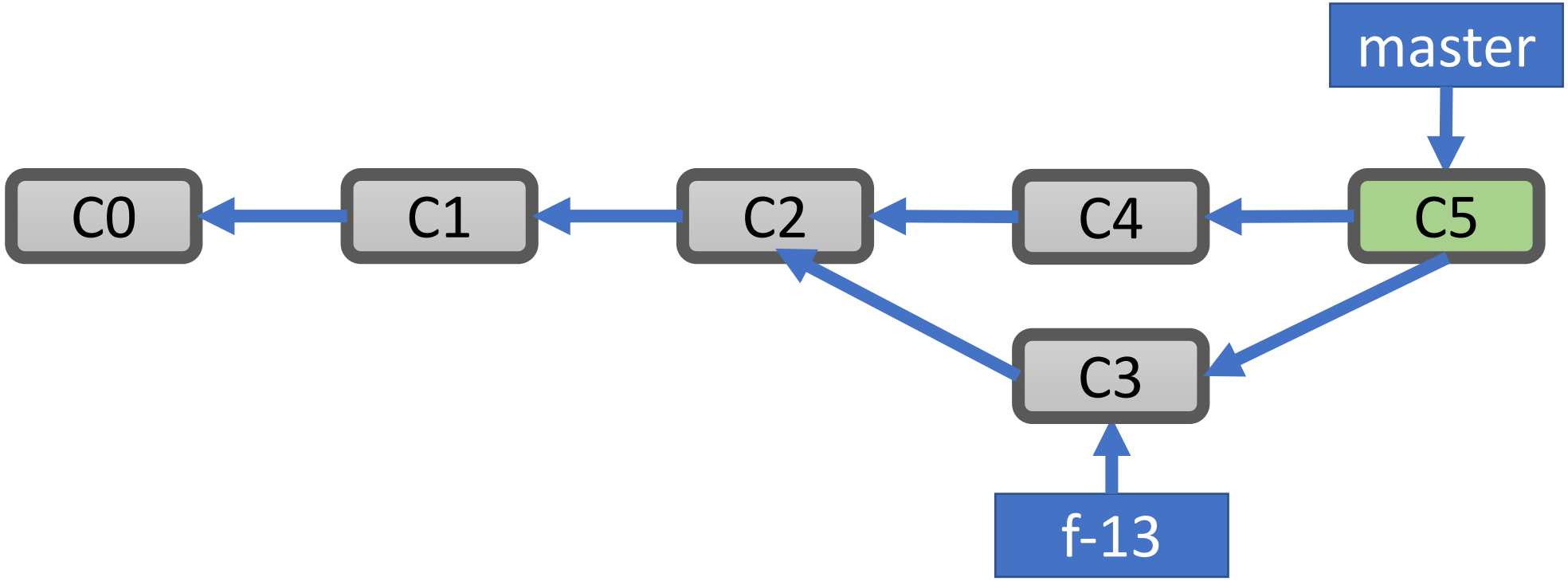
Beipiel Merge Konflikt

- Wenn Du ein grafisches Tool zur Bereinigung benutzen willst, dann verwende **git mergetool**, welches ein passendes grafisches 'merge'-Tool startet und dich durch die Konfliktbereiche führt.
- **Git mergetools** verwendet eingebaute Tools aus denen du eines auswählen kannst:
  - **opendiff kdiff3 tkdiff xxdiff meld tortoisemerge gvimdiff diffuse diffmerge ecmerge p4merge araxis bc3 codecompare vimdiff emerge**
- Ich würde allerdings ein eingebautes Tool in deinem IDE (Eclipse, IntelliJ, PyCharm, etc.) oder TortoiseGit empfehlen.

Beipiel Merge Konflikt

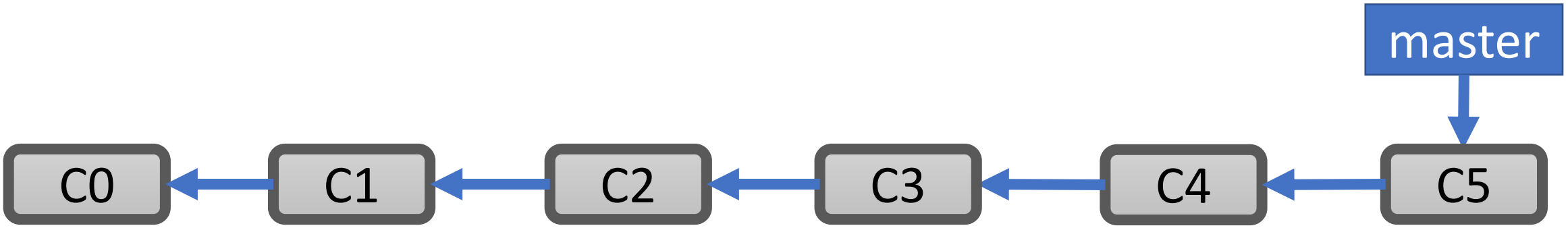
- Du kannst **git status** erneut ausführen, um zu sehen, ob alle Konflikte gelöst sind
- Wenn du zufrieden bist und Du geprüft hast, dass alle Konflikte beseitigt wurden, kannst Du **git commit** ausführen, um den 'merge commit' abzuschließen.

Beipiel Merge Konflikt



Beipiel Merge Konflikt

- Lösche wieder den feature branch mit:
  - **git branch –d f13**



Challenge!



Challenge!

- Dein Kollege ruft dich an dass es mit seinem Feature Branch Probleme gibt. Nachdem du das letzte mal den Hotfix zusammengeführt hattest gibt es jetzt einen Konflikt.
- Es ist dringend.



Aufgabenbeschreibung

- Fork und klone das Flair Projekt von Zalando von dieser Seite (oder aus den Anlagen dieser Lektion):
  - https://github.com/datamics/flair
- Schaue dir die bestehenden Zweige an.
- Merge den **feature-13** Branch in den **master** Branch
- Löse den Merge–Konflikt
- Lade die Änderungen auf das Remote-Repository

Challenge!

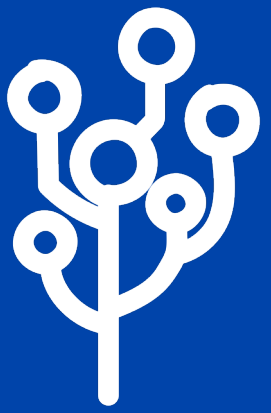
Lösung



Aufgabenbeschreibung

- Fork und klone das Flair Projekt von Zalando von dieser Seite (oder aus den Anlagen dieser Lektion):
  - https://github.com/datamics/flair
- Schaue dir die bestehenden Zweige an.
- Merge den **feature-13** Branch in den **master** Branch
- Löse den Merge–Konflikt
- Lade die Änderungen auf das Remote-Repository

Rebase

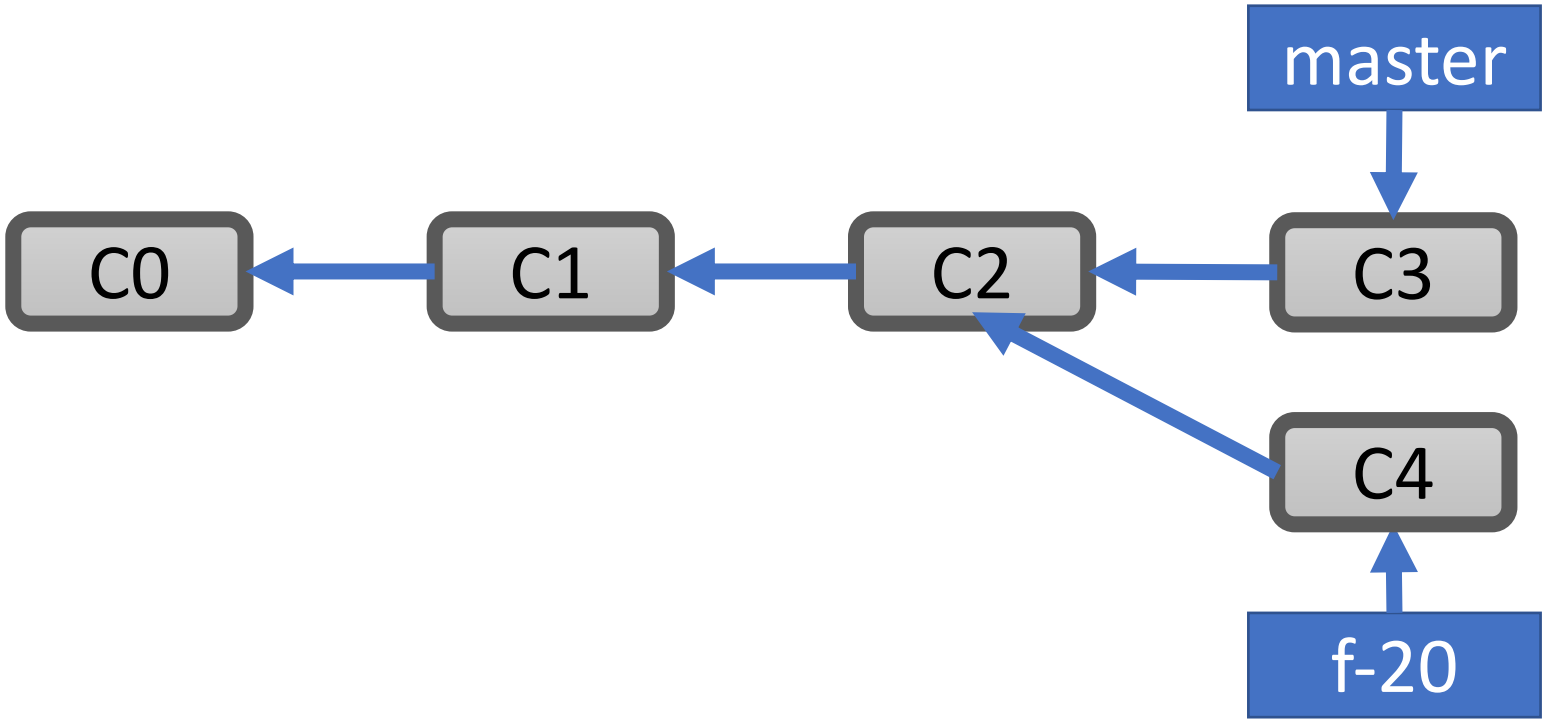


Rebase

- In Git gibt es zwei Möglichkeiten, Änderungen von einem Branch in einen anderen zu integrieren:
  - **Merge** (das Zusammenführen)
  - **Rebase** (das Wiederherstellen)
- In dieser Lektion erfährst du:
  - was Rebasing bedeutet
  - wie es funktioniert
  - warum es ein ziemlich erstaunliches Tool ist
  - welchen Fällen du es nicht verwenden solltest.

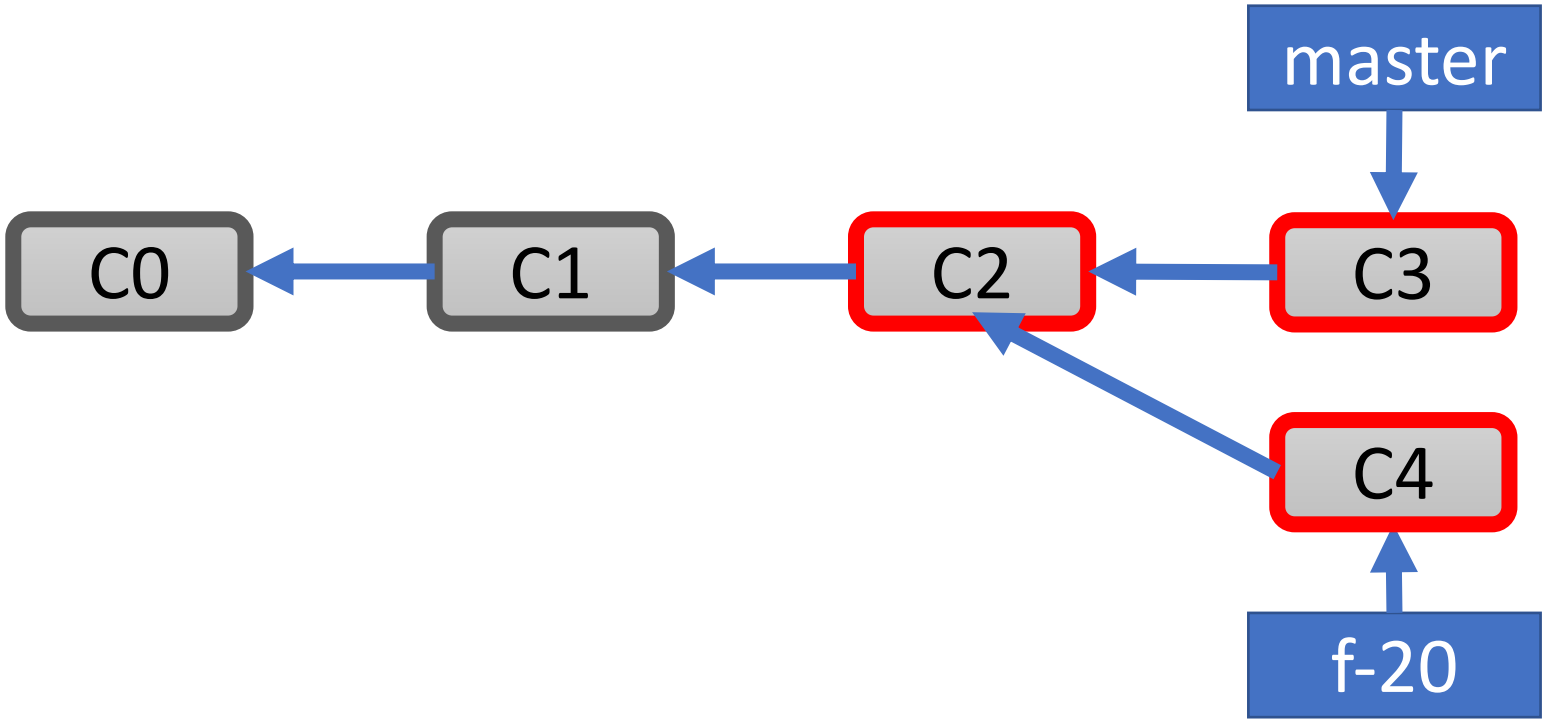
Merge

- Erinnerst du dich noch an das Beispiel aus dem Merging:



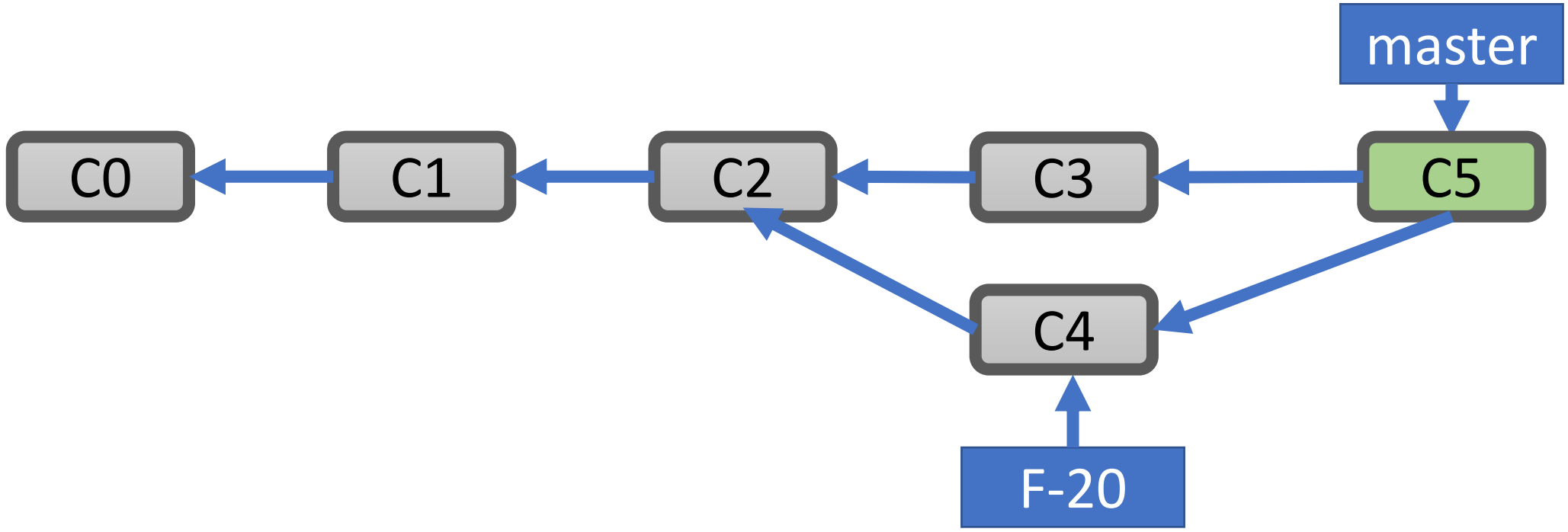
Merge

- Der einfachste Weg, die Branches zu integrieren, wie du bereits gesehen hast, ist der Befehl **merge**.



Beispiel für einfaches Branching und Merging

- Es wird eine Drei-Wege-Merge zwischen den beiden neuesten Zweig-Snapshots durchgeführt:



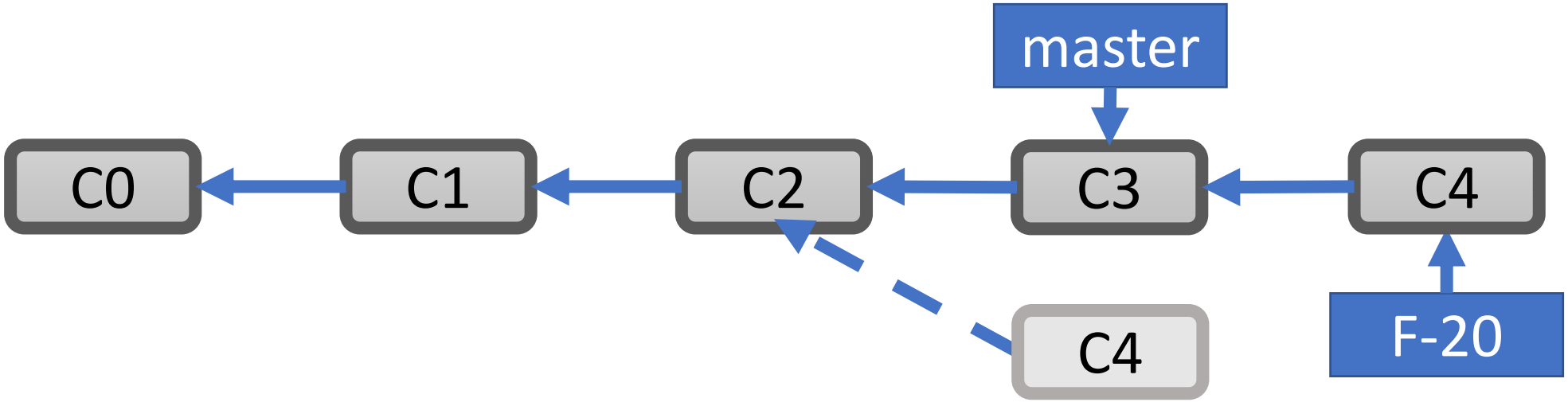
Rebase

- Es gibt jedoch noch eine andere Möglichkeit.
- Du kannst den Patch der in **C4** eingeführten Änderung im **f-20** Branch übernehmen und ihn erneut auf **C3** des **master-Branches** anwenden.
- In Git wird dies als **Rebasing** bezeichnet. Mit dem Befehl **rebase** kannst du alle Änderungen, die in einem Zweig festgeschrieben wurden, in einem anderen Zweig wiedergeben.

Rebase

- Das wäre dann der folgende Code:
  - **git checkout f-20**
  - **git rebase master**

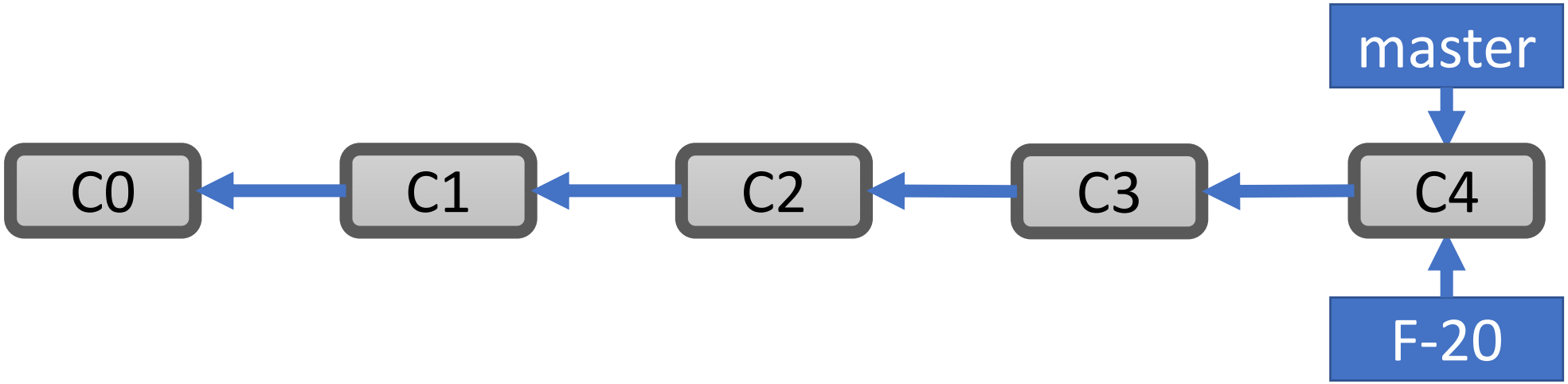
Rebase



Rebase

- Jetzt musst du noch in den **master-Branch** wechseln und einen **fast-forward** Merge durchführen.
  - **git checkout master**
  - **git merge f-20**

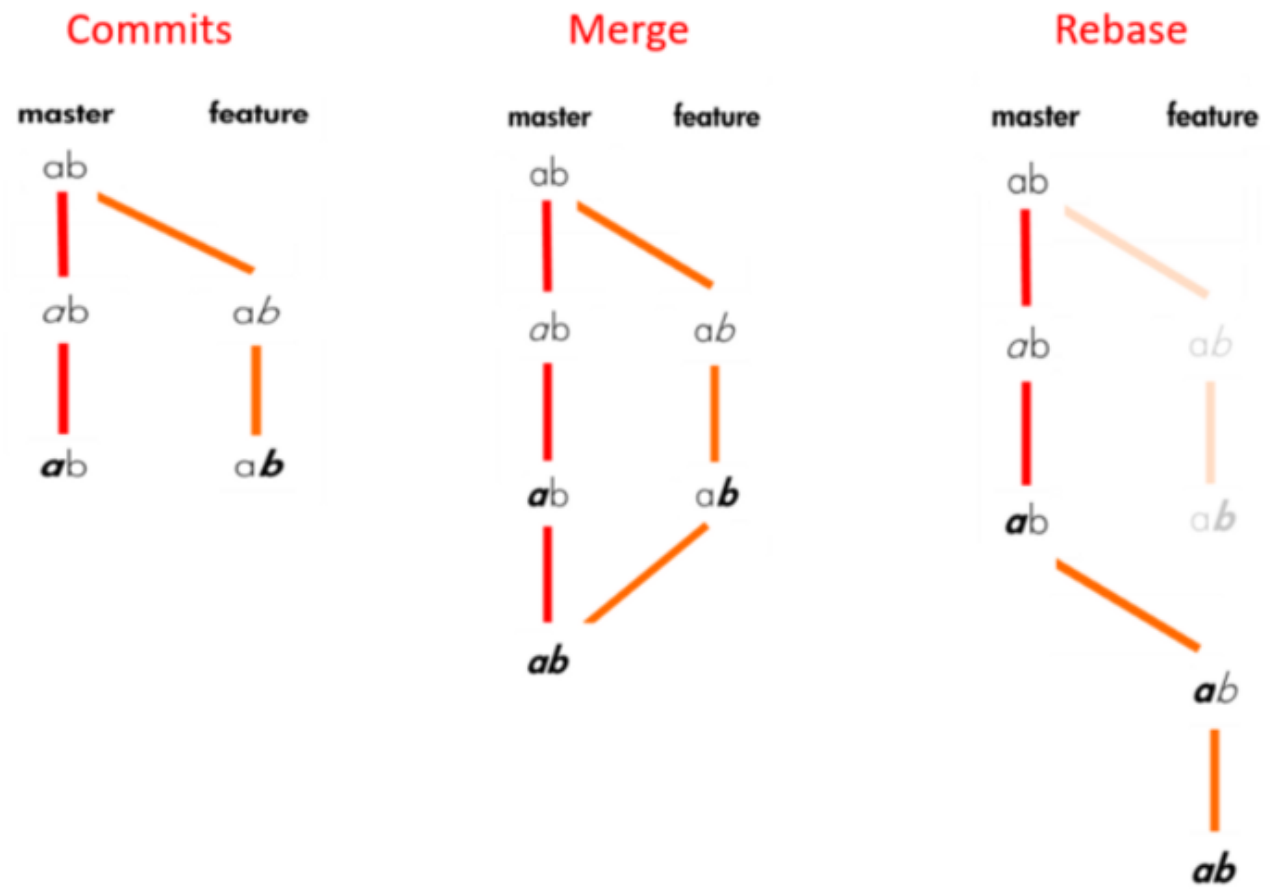
Rebase



Rebase oder Merge?

- Wenn du den Verlauf genau so sehen möchtest, wie er stattgefunden hat, dann solltest du “merge" verwenden.
- Beim Merge wird der Verlauf beibehalten, beim Rebase wird der Verlauf neu erstellt
- Rebase ist besser, um einen komplexen Verlauf mit vielen Branches zu optimieren.

Rebase oder Merge?



Rebase oder Merge?

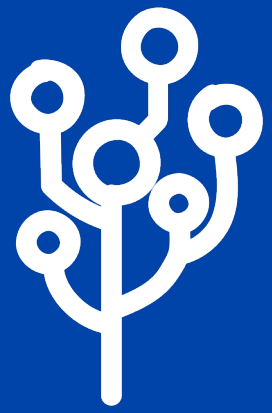
- Durch einen Rebase werden Konflikte jeweils nach einem Commit angezeigt, während durch das Merge alle Konflikte gleichzeitig angezeigt werden
- Mit dem Rebase ist es besser und einfacher, mit Konflikten umzugehen
- Aber wenn es viele Konflikte gibt, ist das Zurücksetzen (**revert**) eines Rebase ist viel schwieriger als das Zurücksetzen eines Merges

Rebase oder Merge?

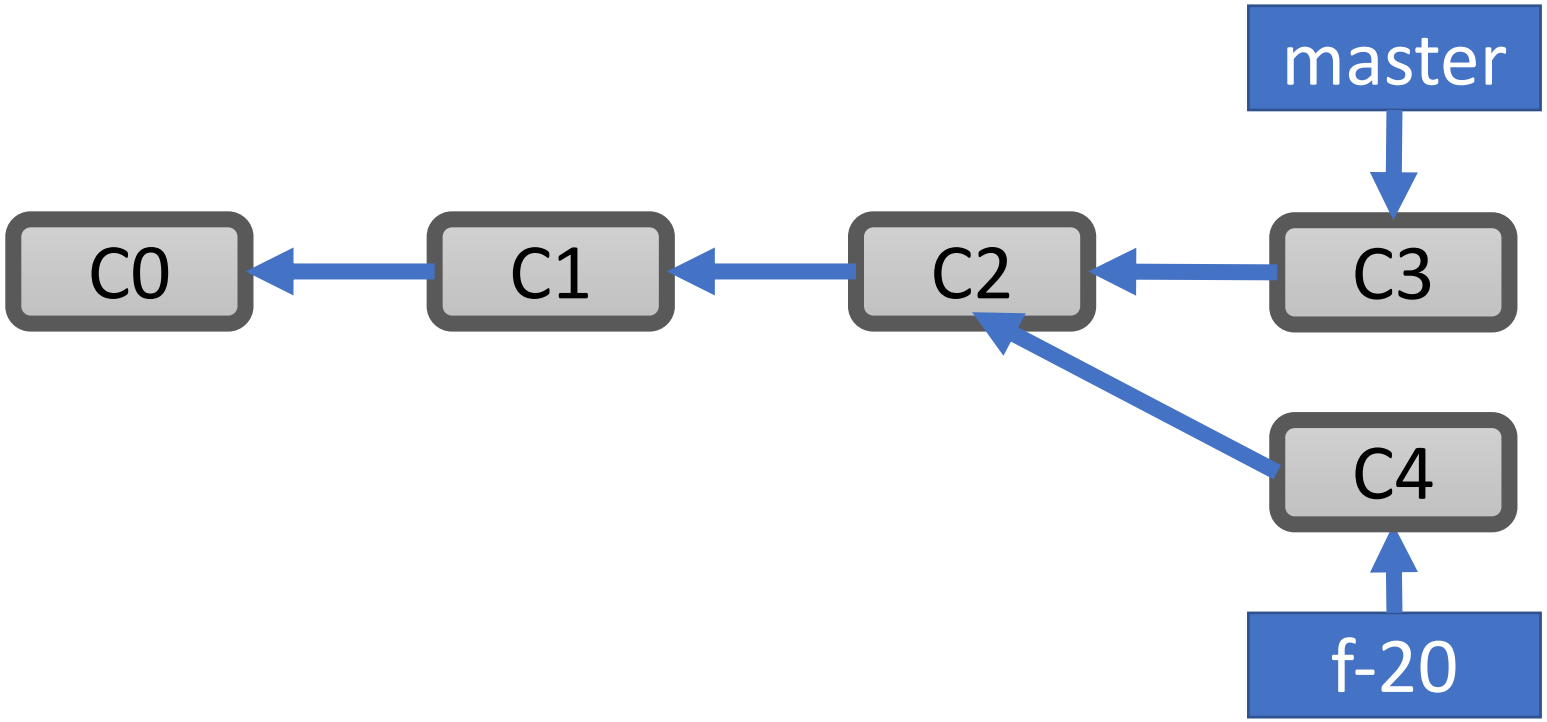
- Wenn der Feature-Branch, von dem du die Änderungen erhältst, mit anderen Entwicklern geteilt wird, wird ein erneuter Rebase nicht empfohlen, da beim erneuten Rebase inkonsistente Repositorys erstellt werden. Für den Einzelnen macht das Rebase viel Sinn.
- Die **goldene** Regel für Git-Rebase ist, es niemals in öffentlichen Branches zu verwenden.

Rebase

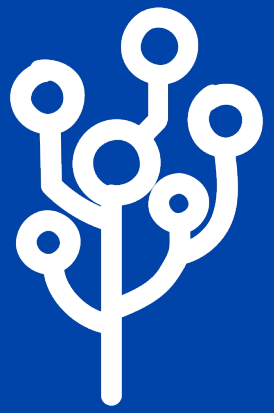
Teil 2



Rebase



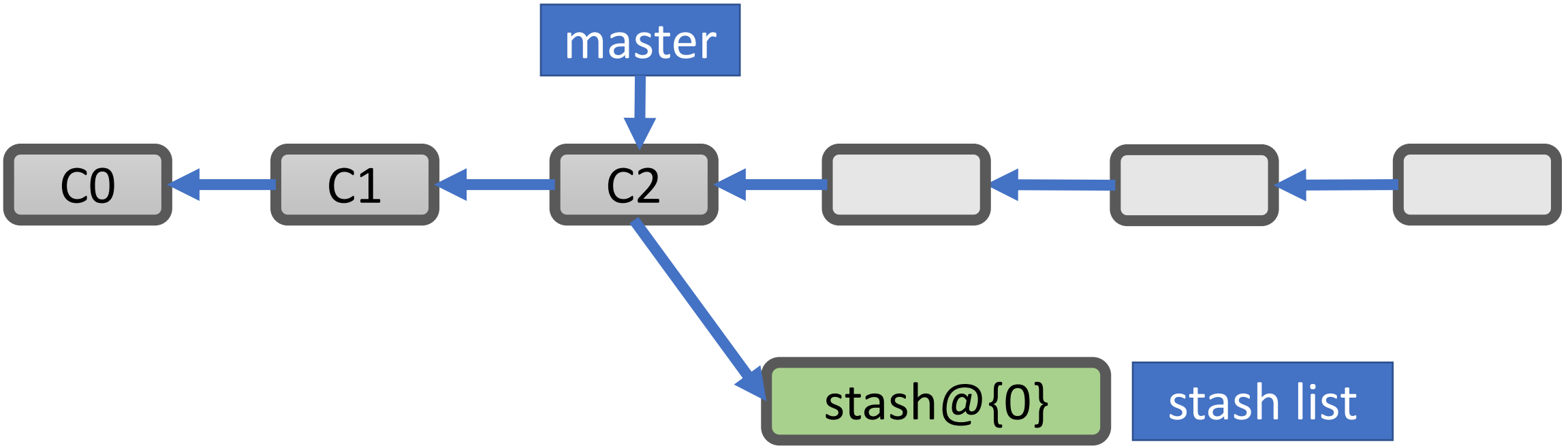
Git stash



Git Stash

- Verwende **git stash**, wenn du den aktuellen Status des Arbeitsverzeichnisses und des Index aufzeichnen, aber später zu einem sauberen Arbeitsverzeichnis zurückkehren möchtest.
- Der Befehl speichert deine lokalen Änderungen und setzt das Arbeitsverzeichnis zurück, damit es mit dem HEAD-Commit übereinstimmt.

Git Stash



Git Stash

- Die von diesem Befehl verwahrten Änderungen können mit der **git stash list** aufgelistet, mit **git stash show** überprüft und mit **git stash apply** wiederhergestellt (über einen anderen Commit hinweg) werden.
- Das Aufrufen von **git stash** ohne Argumente entspricht **git stash push**.
- Ein Stash wird standardmäßig als "WIP on branchname ..." aufgelistet.

Git Stash

- Stash folgt dem Prinzip eines Stapels (Stack):

